

SSN College of Engineering

LABORATORY RECORD

Subject: Web Application Development Laboratory

Exercise 4: CRUD Operations using Spring Boot, MongoDB and Postman

SUBMITTED BY

Name: Vijayaraaghavan K S

Register Number: 3122247001066

Branch: Department of CSE

Semester / Year: V Semester / III Year

Subject Code: ICS1511 Batch: 2024–2029

1. Problem Description

Scenario

Build a REST API using Spring Boot that performs CRUD (Create, Read, Update, Delete) operations on an **Employee** record, backed by MongoDB. The API is built with a Controller → Service → Repository layered flow and tested with request/response evidence for every endpoint.

MongoDB Collection

- **employees** – id, name, department, salary

Required REST APIs

Operation	Method	URL
Create Employee	POST	/api/employees
Read All Employees	GET	/api/employees
Read One Employee	GET	/api/employees/{id}
Update Employee	PUT	/api/employees/{id}
Delete Employee	DELETE	/api/employees/{id}

Technology Stack

- Spring Boot 4.1.1 (Java 17 language level, run on a JDK 21 runtime)
- spring-boot-starter-web, spring-boot-starter-data-mongodb, spring-boot-starter-validation, Lombok
- MongoDB (local instance, database EmployeeDB)
- Maven (via the Maven Wrapper, ./mvnw) for build/dependency management

Request Flow

```
Client --POST /api/employees--> EmployeeController
      --> EmployeeService --> EmployeeRepository --> MongoDB
      <-- saved Employee (JSON) <--
```

Each HTTP verb maps to one controller method, which delegates to EmployeeService (business logic layer), which in turn calls EmployeeRepository (a MongoRepository<Employee, String>) to talk to MongoDB.

2. Source Code

File: application.properties

```
spring.application.name=employee-crud
spring.mongodb.uri=mongodb://localhost:27017/EmployeeDB
server.port=8080
```

Note: Spring Boot 4 renamed the MongoDB connection property prefix from `spring.data.mongodb` (used in Boot 3.x) to `spring.mongodb` — discovered by inspecting `MongoProperties` in the `spring-boot-mongodb` module, since using the old key silently fell back to the default test database.

File: Employee.java (Model)

```
package com.ssn.employeecrud.model;

import jakarta.validation.constraints.NotBlank;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

// Maps to the "employees" collection in MongoDB. @Id becomes the document's
// _id.
@Data
@NoArgsConstructor
@AllArgsConstructor
@Document(collection = "employees")
public class Employee {

    @Id
    private String id;

    @NotBlank(message = "Name cannot be empty")
    private String name;

    private String department;

    private double salary;
}
```

File: EmployeeRepository.java

```
package com.ssn.employeecrud.repository;

import com.ssn.employeecrud.model.Employee;
import org.springframework.data.mongodb.repository.MongoRepository;

// Extending MongoRepository gives save(), findAll(), findById(), deleteById()
// for free.
public interface EmployeeRepository extends MongoRepository<Employee, String>
{
}
```

File: EmployeeService.java

```
package com.ssn.employeecrud.service;

import com.ssn.employeecrud.model.Employee;
import com.ssn.employeecrud.repository.EmployeeRepository;
import org.springframework.stereotype.Service;

import java.util.List;

// Sits between the controller and the repository. Kept thin here since the
// exercise only needs plain CRUD, but this is where business logic would go.
@Service
public class EmployeeService {

    private final EmployeeRepository repository;

    public EmployeeService(EmployeeRepository repository) {
        this.repository = repository;
    }

    public Employee create(Employee employee) {
        return repository.save(employee);
    }

    public List<Employee> getAll() {
        return repository.findAll();
    }

    public Employee getById(String id) {
        return repository.findById(id).orElse(null);
    }

    public Employee update(String id, Employee updated) {
        return repository.findById(id).map(employee -> {
            employee.setName(updated.getName());
            employee.setDepartment(updated.getDepartment());
            employee.setSalary(updated.getSalary());
        });
    }
}
```

```

        return repository.save(employee);
    }).orElse(null);
}

public void delete(String id) {
    repository.deleteById(id);
}
}

```

File: EmployeeController.java

```

package com.ssn.employeecrud.controller;

import com.ssn.employeecrud.model.Employee;
import com.ssn.employeecrud.service.EmployeeService;
import jakarta.validation.Valid;
import org.springframework.web.bind.annotation.*;

import java.util.List;

// REST flow: Postman request -> this Controller -> EmployeeService ->
// EmployeeRepository -> MongoDB.
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    private final EmployeeService service;

    public EmployeeController(EmployeeService service) {
        this.service = service;
    }

    // CREATE: POST /api/employees with JSON body
    @PostMapping
    public Employee createEmployee(@Valid @RequestBody Employee employee) {
        return service.create(employee);
    }

    // READ ALL: GET /api/employees
    @GetMapping
    public List<Employee> getAllEmployees() {
        return service.getAll();
    }

    // READ ONE: GET /api/employees/{id}
    @GetMapping("/{id}")
    public Employee getEmployee(@PathVariable String id) {
        return service.getById(id);
    }

    // UPDATE: PUT /api/employees/{id} with JSON body of updated fields
    @PutMapping("/{id}")

```

```

public Employee updateEmployee(@PathVariable String id, @Valid
    @RequestBody Employee employee) {
    return service.update(id, employee);
}

// DELETE: DELETE /api/employees/{id}
@DeleteMapping("/{id}")
public String deleteEmployee(@PathVariable String id) {
    service.delete(id);
    return "Employee deleted";
}
}

```

3. API Testing

The application was run locally (java -jar target/employee-crud-0.0.1-SNAPSHOT.jar, listening on http://localhost:8080) with a local MongoDB instance. Each endpoint below was exercised with curl — a command-line REST client that sends the exact same HTTP requests Postman would, and is shown here in place of Postman GUI screenshots.

a. Create – POST /api/employees

Request:

```

curl -X POST http://localhost:8080/api/employees \
-H "Content-Type: application/json" \
-d '{"name":"Arjun Kumar","department":"CSE","salary":55000}'

```

Response:

```

{"id":"6ab107fdc65399dd35c6425a","name":"Arjun
    Kumar","department":"CSE","salary":55000.0}

```

A second employee was created the same way:

```

{"id":"6ab107fdc65399dd35c6425b","name":"Priya
    Sharma","department":"IT","salary":60000.0}

```

b. Read All – GET /api/employees

```

curl http://localhost:8080/api/employees

```

```
[
  {"id":"6ab107fdc65399dd35c6425a","name":"Arjun
    Kumar","department":"CSE","salary":55000.0},
  {"id":"6ab107fdc65399dd35c6425b","name":"Priya
    Sharma","department":"IT","salary":60000.0}
]
```

c. Read One – GET /api/employees/{id}

```
curl http://localhost:8080/api/employees/6ab107fdc65399dd35c6425a
```

```
{"id":"6ab107fdc65399dd35c6425a","name":"Arjun
  Kumar","department":"CSE","salary":55000.0}
```

d. Update – PUT /api/employees/{id}

```
curl -X PUT http://localhost:8080/api/employees/6ab107fdc65399dd35c6425a \
-H "Content-Type: application/json" \
-d '{"name":"Arjun Kumar","department":"CSE","salary":58000}'
```

```
{"id":"6ab107fdc65399dd35c6425a","name":"Arjun
  Kumar","department":"CSE","salary":58000.0}
```

The salary was updated from 55000 to 58000 while the id stayed the same.

e. Delete – DELETE /api/employees/{id}

```
curl -X DELETE http://localhost:8080/api/employees/6ab107fdc65399dd35c6425a
```

Employee deleted

f. Read All (after delete) – confirms the delete

```
curl http://localhost:8080/api/employees
```

```
[{"id":"6ab107fdc65399dd35c6425b","name":"Priya
  Sharma","department":"IT","salary":60000.0}]
```

Only Priya Sharma remains, confirming the delete removed exactly the targeted document from MongoDB.

4. Learning Outcomes

This exercise built a complete REST backend using Spring Boot layered into Controller, Service and Repository, backed by MongoDB through Spring Data. A significant real debugging lesson came from a Spring Boot version upgrade: in Spring Boot 4.1.1 the MongoDB connection property prefix changed from `spring.data.mongodb` to `spring.mongodb`, so using the older key silently connected to the default test database instead of the intended one — this was diagnosed by inspecting the `spring-boot-mongodb` module's `MongoProperties` class directly. The exercise also reinforced how each HTTP verb (POST/GET/PUT/DELETE) maps cleanly onto one controller method and one repository operation, and how `curl` can be used as a lightweight, scriptable substitute for manual Postman testing.